They started with the business success metrics they wanted to get out of the feature they were delivering. In this case the business success metrics they came up with were as follows:

- ***Items sold*** —They were already tracking this as a success metric of the site in general.
- ***Number of items per order*** —If this feature was successful, they would see this number going up. This was the key metric that related to the strategic goal laid out by the leadership team.
- ***Recommended items per order*** —This was the key metric that showed the development team how effective their feature was. If this number was going up, then the product recommender was a success. If they noticed this number increasing at the same rate as the number of items per order, then they would know that this was the driving feature for the executive goal.

Like the team in chapter 6, they used StatsD to instrument their code and send these metrics back to their monitoring system, as shown in the following listing.

**Listing 9.1. Using StatsD to add business-specific metrics to the code**

```
private static final StatsDClient statsd = new
     NonBlockingStatsDClient("the.prefix", "statsd-host", 8125);

...

    statsd.incrementCounter("orders");
    statsd.recordGaugeValue("itemsPerOrder", x);
    statsd.recordGaugeValue("recommendedItems", y);
```

Incrementing the counter for total orders

Setting up the StatsD client

Setting item count per order for this order

How many items in this order were from recommendations

Different teams had different release metrics. The web services team was working on improving their pull request workflow, so they focused on PR comments and pull requests/commits to make sure that was healthy. This is shown in figure 9.17.

**Figure 9.17. The web services team's dashboard. Development time, PR comments, and PR hours open are right on target. The pulse at the bottom shows that SCM activity over time is healthy.**